9

BÖLÜM: KUBERNETES ILE TANIŞMA – ORKESTRA ŞEFI | MODULE: INTRODUCTION TO KUBERNETES – THE CONDUCTOR

En Temel Atom: Pod Nedir? Neden Doğrudan Konteyner Değil?

The Atomic Unit: What is a Pod and Why Not Bare Containers?

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

K8s dünyasında tek başına konteyner kiralayamazsınız; en küçük kiralık daire 'Pod'dur! Pod bir bezelye kabuğu, konteynerler ise içindeki bezelyelerdir. Çoğunlukla bir pod'da 1 bezelye (örneğin Spring Boot) yaşar. Ama bazen ana uygulamanın yanına ona yardımcı olan bir oda arkadaşı (Sidecar Konteyneri: örneğin log toplayıcı veya güvenlik ajanı) koyarsınız. Aynı pod'daki konteynerler aynı banyoyu (IP adresi) ve aynı buzdolabını (ortak disk) paylaşır!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

In Kubernetes, you cannot rent a bare container; the smallest rentable apartment is a 'Pod'! A Pod is a pea pod, and containers are the peas inside. Usually, a pod hosts 1 main pea (your Spring Boot microservice). But sometimes, you pair it with a helpful roommate (Sidecar container: log shipper, envoy proxy). All containers in the same Pod share the exact same apartment IP and localhost loopback!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Pod, bir veya daha fazla konteynerin aynı Linux Network Namespace'ini, IPC namespace'ini ve depolama birimlerini paylaştığı soyut bir yapıdır. Bir pod ayağa kalktığında K8s önce 'pause' adında minik bir altyapı konteyneri başlatır. Bu pause konteyneri IP adresini ve ağ arayüzünü alır. Ardından gelen Spring Boot konteyneri ve sidecar konteynerleri bu ağ alanına katılır (`--net=container:pause`). Bu yüzden aynı pod içindeki iki farklı konteyner birbiriyle `localhost:port` üzerinden sıfır ağ gecikmesiyle konuşabilir.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

A Pod encapsulates one or more co-located containers sharing Linux IPC, UTS, and Network namespaces via a lightweight 'pause' container. When a Pod initializes, the pause container claims the Pod IP. Subsequent app and sidecar containers attach to this namespace, allowing them to communicate over `localhost` with zero network serialization overhead and shared volumes.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```yaml
# Tek Pod içinde 2 konteyner örneği (Spring Boot + Log Tailer Sidecar):
apiVersion: v1
kind: Pod
metadata:
  name: coffee-with-sidecar
spec:
  containers:
    - name: spring-app
      image: ghcr.io/eli5-k8s/spring-boot-k8s-eli5:1.0.0
      ports:
        - containerPort: 8080
    - name: log-shipper-sidecar
      image: alpine:latest
      command: ["sh", "-c", "while true; do echo 'Sidecar logging...'; sleep 10; done"]
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

İki bağımsız iş yapan mikroservisi asla aynı pod'un içine koymayın! Pod'lar birlikte ölür ve birlikte ölçeklenir. Sadece birbirinden ayrılamaz derecede kenetlenmiş yardımcı araçlar (proxy, log forwarder) sidecar yapılmalıdır.

🇬🇧 WATCH OUT

Never combine two independent microservices inside the same Pod! Containers in a pod scale together. Use sidecars strictly for tightly coupled helpers.

🎯 Özet Çıkarımlar

🎯 Key Takeaways

- Pod, Kubernetes'in zamanladığı ve yönettiği en küçük konuşlandırma birimidir.
- Aynı Pod içindeki konteynerler localhost üzerinden haberleşir ve IP paylaşır.
- Çoğu senaryoda 1 Pod = 1 Uygulama Konteyneri kuralı geçerlidir.

- A Pod is the smallest atomic unit of scheduling in Kubernetes.
- Containers inside the same Pod share an IP address and reach each other on localhost.
- The standard pattern is 1 App Container per Pod, unless utilizing a Sidecar.